

Plan Rozwoju i Podział Pracy — AI Traffic Prediction

Dokument zawiera analizę obecnego stanu projektu, strategię włączenia darmowych źródeł danych, plan ulepszenia predykcji oraz szczegółowy podział pracy dla 3-osobowego zespołu deweloperskiego.

I. Darmowe źródła danych do integracji

Zgodnie z weryfikacją, oto plan dla dodatkowych danych, ułożony od najwyższego priorytetu:

1. **Cechy temporalne (godzina, dzień tygodnia, święta)**
2. **Wpływ:** Bardzo wysoki. Podstawowa metryka wzorców ruchu.
3. **Priorytet:** P1 (do wdrożenia natychmiast).
4. **Implementacja:** Generowanie czasu lokalnie (`datetime`) z użyciem funkcji cyklicznych (sin/cos). Święta (PublicHolidays API) do cache raz dziennie.
5. **Historyczne wzorce tego samego punktu (Profil tygodniowy)**
6. **Wpływ:** Bardzo wysoki. Pozwala modelowi bazować na różnicy między stanem aktualnym a normą dla danego momentu.
7. **Priorytet:** P1.
8. **Implementacja:** Wydłużenie retencji SQLite (w `app/storage.py`) z 24h na 14-28 dni. Funkcja agregująca medianę z historii.
9. **Pogoda (Open-Meteo API)**
10. **Wpływ:** Wysoki. Kluczowe dla trudnych warunków atmosferycznych.
11. **Priorytet:** P2.
12. **Implementacja:** Polling co 30 min (nowy job w `app/scheduler.py`). Cechy: `is_raining` , `wind_speed` , `visibility` . Brak wymogu klucza API.
13. **AISStream.io i Harmonogramy statków (MarineTraffic/Porty)**
14. **Wpływ:** Średni/Wysoki (zależnie od portu).
15. **Priorytet:** P2/P3.
16. **Implementacja:** Aplikacja ma już fundament w `ais_client.py` . Należy dopracować wpływ statku na score (np. `large_vessel_arrival_soon`) i dodać proste reguły ręczne powtarzalnych rejsów promów do bazy.
17. **Planowane roboty drogowe (GDDKiA/ZDiZ)**
18. **Wpływ:** Niski/Średni.
19. **Priorytet:** P3 (Później).
20. **Implementacja:** Utrzymanie scrapera bywa kosztowne, początkowo opieramy się na informacjach typu `roadworks` bezpośrednio z payloadu TomTom.

II. Główne cele rozwoju aplikacji

Aplikacja ewoluuje z prostego dashboardu dla jednego odbiorcy w kompleksowe narzędzie Logistyczno-Drogowe, obsługujące trzy osoby: 1. **Kierowcy** – Czyste, zwięzłe rekomendacje i alerty głosowe w trasie. 2. **Dyspozytorzy** – Złożone mapy i predykcje,

zarządzanie flotą i portami. 3. **Klienci firm** – Ograniczony widok statusu ("Czy transport będzie na czas?") z opcją pobrania raportu opóźnienia.

III. Analiza projektu

Obecny stos technologiczny i struktura: * **Backend:** FastAPI (Python), SQLite, `apscheduler` do zadań w tle (TomTom, Tristar), WebSockets (AISStream). * **Frontend:** Vanilla JavaScript (`app.js`), HTML, CSS bezpośrednio w podkatalogu `app/static` . Brak bundlera (jak Webpack/Vite) oraz brak frameworka (React/Vue). * **Integracje:** TomTom API, TRISTAR API, Groq LLM (raporty), AISstream.io.

Obecne miejsca logiki: * **Live Traffic / TomTom:** `app/tomtom.py` , `app/tristar.py` * **Predykcje opóźnień:** `app/analysis.py` (obecnie podstawowa regresja liniowa krótkiego horyzontu na danych bieżących). * **Raporty:** `app/reports.py` oraz `app/groq_client.py` (LLM). * **UI i Wykresy:** Czysty plik `app/static/app.js` (używa Leaflet do mapy i Chart.js do wykresów).

Wnioski architektoniczne: Głównym wąskim gardłem przed wprowadzeniem zaawansowanych paneli (dashboard firmowy, role) jest frontend pisany w Vanilla JS. Wprowadzanie powiadomień push, service workerów i ról w jednym pliku `app.js` doprowadzi do chaosu i konfliktów w kodzie. Z poziomu backendu, architektura rozbita na moduły FastAPI jest w miarę stabilna, chociaż brakuje podziału na warstwę usług (`services/`).

IV. Roadmapa Rozwoju

- **Etap 1:** Analiza projektu i reorganizacja struktury.
- **Etap 2:** Ekran startowy i wdrożenie 3 ról użytkownika (Kierowca / Dyspozytor / Klient).
- **Etap 3:** Poprawa UI i widoków dla ról.
- **Etap 4:** Cechy temporalne (Dni wolne, godziny szczytu) do backendu.
- **Etap 5:** Wdrożenie profilu historycznego punktów.
- **Etap 6:** Nowy moduł pogodowy (Open-Meteo) połączony z silnikiem predykcji.
- **Etap 7:** Ewolucja silnika rekomendacji i powodów opóźnień.
- **Etap 8:** Raporty opóźnień (Eksport PDF/CSV).
- **Etap 9:** Analityka firmowa w nowym UI (Wykresy firmowe).
- **Etap 10:** Dane portowe i statki (Dokończenie reguł AIS).
- **Etap 11:** PWA i Web Push Notifications.
- **Etap 12:** Opcjonalne alerty ElevenLabs.
- **Etap 13:** Testy i prezentacja.

Co robimy teraz: Role UI, Wykresy, Cechy czasowe i profil historyczny punktów, proste rekomendacje, raporty opóźnień (PDF). **Co zostawiamy na później:** Zaawansowane modele ML, ElevenLabs, webowe powiadomienia Push z ServiceWorkerem, pełna integracja AISStream.

V. Proponowana struktura folderów i refaktor

Sugerowana bezpieczna transformacja, tak by nie zepsuć obecnego projektu z dnia na dzień:

```
/backend (wydzielone z obecnego folderu app)
  /api/ (routery endpointow)
  /services/
    weather/
    aisstream/
    recommendations/
    reports/
    elevenlabs/
```

```
historical-profile/  
/db/ (storage.py i sqlite)  
/jobs/ (scheduler.py)  
  
/frontend (Sugerowany nowy projekt np. Vite + React / TypeScript)  
/src/  
  /features/  
    /roles/  
    /driver/  
    /dispatcher/  
    /client-tracking/  
    /company-analytics/  
    /reports/  
    /recommendations/  
    /notifications/  
  /shared/  
    /components/  
    /utils/  
  /api/
```

(W przypadku pozostania przy Vanilla JS ze względu na wymóg Nie przepisuj całego projektu od zera , folder frontend zastępujemy folderem src/ z modułami .js obsługiwanymi przez ES Modules lub Webpacka).

VI. Ulepszenia Predykcji (Scoring)

Obecny kod w app/analysis.py liczy proste wzrosty kongestii. Zostanie dodany prosty algorytm wzorcowy przed użyciem pełnego modelu ML: delay_risk_score = (traffic_ratio_live * 0.4) + (historical_anomaly * 0.3) + (weather_penalty * 0.1) + (port_ship_penalty * 0.1) + (temporal_peak_penalty * 0.1)

Zmapowane na poziomy ryzyka: * 0-30 Niskie: "Jedź teraz - brak opóźnień" * 31-60 Średnie: "Zmień trasę" * 61-80 Wysokie: "Poczekaj w strefie buforowej" * 81-100 Bardzo wysokie: "Wysokie ryzyko opóźnienia — firma powinna powiadomić klienta"

VII. Podział Pracy dla 3 Osób i Zasady Git

Zasady pracy (Git Rules): 1. Każda osoba ma przypisane oddzielne branch'e w formacie feature/nazwa . 2. Zakaz modyfikacji głównych punktów styku (np. app/main.py) bez komunikacji. 3. Przed pushem zawsze robimy git pull --rebase origin main . 4. Modyfikacje wspólnych typów i logiki TomTom podlegają bezwzględnemu PR.

Podział Zadań

Osoba	Zakres Odpowiedzialności	Edytowane Foldery/Pliki	Pliki, których nie edytować	Zadania Konkretnie	Priorytet	Ryzyko Konfliktów
Osoba 1	Interfejs UI / Role / Widoki	src/features/roles , src/features/driver , src/features/dispatcher	Skrypty analityczne app/analysis.py , baza SQLite	Ekran powitalny ról. Widok Kierowcy (jasny UX), Dyspozytora i Klienta. Przygotowanie pod PWA.	P1	Niskie (osobny obszar frontendowy)
Osoba 2	Raporty (PDF) / Powiadomienia /	app/reports.py , backend/services/reports ,	Pliki ról (Osoba 1), Logika pogody	PDF Raport Opóźnienia z	P1	Średnie (styki na API)

Osoba	Zakres Odpowiedzialności	Edytowane Foldery/Pliki	Pliki, których nie edytować	Zadania Konkretne	Priorytet	Ryzyko Konfliktów
	Powody	moduły powiadomień	(Osoba 3)	bazy, rozbudowa cause o nowe źródła danych (współpraca z backendem), system alertów push (planowanie).		
Osoba 3	Analityka Firmowa / Wykresy / Cechy Czasowe / Rekomendacje	app/analysis.py , app/storage.py , backend/services/weather , UI Analityki	System raportowania (Osoba 2), UI ról (Osoba 1)	Integracja Open-Meteo, kalendarz świąt, implementacja scoringu ulepszonej predykcji, agregaty SQLite dla analityki firm (Wykresy w dyspozytorni).	P1	Średnie (modyfikacja schematu SQLite)

Instrukcja generowania PDF:

Powyższy plik został zapisany w formacie Markdown. Aby zamienić go na PDF użyj jednej z poniższych metod: 1. **W VS Code:** Zainstaluj darmowe rozszerzenie "Markdown PDF" i kliknij na ten plik prawym przyciskiem myszy -> "Export Markdown to PDF". 2. **Narzędzie Pandoc:** W terminalu uruchom: `pandoc docs/plan-podzial-pracy-i-ulepszenia.md -o docs/plan-podzial-pracy-i-ulepszenia.pdf` . 3. **Przez przeglądarkę:** Użyj narzędzia online "Markdown to PDF" lub otwórz plik markdown w przeglądarce i użyj "Drukuj do PDF" (Ctrl+P / Cmd+P).